Mapúa University
School of Information Technology
Year 3 Term 1 AY 2026 - 2027

Final Project Documentation
CSS182-02 - Data Science 2

Real-Time Data Pipeline Project
(DogeCoin x Solana Live Crypto)

Submitted by:
Floranda, Ramuell Franz C.
Santos, Lebbeus Ryan V.

Submitted to:
Dr. John Paul Q. Tomas

Date:
October 12, 2026

1. Live Data Source

The real-time data pipeline utilizes the public CoinGecko API to fetch live market updates for Dogecoin and Solana cryptocurrencies (CoinGecko, n.d.). Data ingestion occurs continuously at specified intervals to capture spot prices, trading volumes, and 24-hour price fluctuations.

The live data source adheres to a strict six-field JSON schema. The crypto_id field (string) uniquely identifies the cryptocurrency using a slug identifier (e.g., "dogecoin" or "solana"), while symbol (string) represents its official ticker symbol (e.g., "DOGE" or "SOL"). The price_usd field (double) records the spot price in US Dollars, and volume_24h (double) tracks the global 24-hour trading volume in USD. The change_24h field (double) indicates the percentage price variation over the last 24 hours rounded to two decimal places. Finally, the timestamp field (string) records the exact data compilation time formatted as YYYY-MM-DD HH:MM:SS.

The specific CoinGecko API endpoints used for data polling are listed below:

1. Dogecoin API Endpoint:
   https://api.coingecko.com/api/v3/simple/price?ids=dogecoin&vs_currencies=usd&include_24hr_vol=true&include_24hr_change=true
2. Solana API Endpoint:
   https://api.coingecko.com/api/v3/simple/price?ids=solana&vs_currencies=usd&include_24hr_vol=true&include_24hr_change=true

2. Kafka Producers and Topics

Apache Kafka serves as the distributed event streaming platform for message ingestion. Independent Python scripts query the CoinGecko API every 10 seconds (subject to CoinGecko rate limits), construct the standardized JSON payload, and serialize the messages before publishing them to dedicated Kafka topics (Apache Software Foundation, n.d.-b).

Listing 1. Python Producer Script for Dogecoin (producer_doge.py)

```
import time
import json
import requests
from datetime import datetime
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

URL =
"https://api.coingecko.com/api/v3/simple/price?ids=dogecoin&vs_currencies=usd&include_24hr_vol=true&inclu
de_24hr_change=true"
TOPIC = "crypto-dogecoin"

print("--- Starting Dogecoin Kafka Producer ---")

while True:
    try:
        res = requests.get(URL, timeout=10)
        if res.status_code == 200:
            data = res.json().get("dogecoin", {})
            if "usd" in data:
                payload = {
                    "crypto_id": "dogecoin",
                    "symbol": "DOGE",
                    "price_usd": float(data["usd"]),
                    "volume_24h": float(data.get("usd_24h_vol", 0.0)),
                    "change_24h": round(float(data.get("usd_24h_change", 0.0)), 2),
                    "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
                }
                producer.send(TOPIC, payload)
                print(f"[DOGE Sent] {payload}")
            else:
                print(f"[DOGE Warning] Missing payload data: {res.text}")
        elif res.status_code == 429:
            print("[DOGE Rate Limit] CoinGecko rate limit reached. Waiting...")
```

```
    else:
        print(f"[DOGE Error] HTTP {res.status_code}")
  except Exception as e:
    print(f"[DOGE Connection Error] {e}")

  time.sleep(10)
```

```
ramuell@LAPTOP-P2C0IDMV:/mnt/c/Users/Ramuell$ docker exec -it kafka kafka-topics --create --topic crypto-dogecoin --boot
strap-server localhost:9092 --partitions 1 --replication-factor 1 --if-not-exists
Created topic crypto-dogecoin.
```

Figure 1. Dogecoin Kafka Topic Creation

The terminal output confirms the successful creation of the crypto-dogecoin Kafka topic. It is configured with one partition and a replication factor of one.

Listing 2. Python Producer Script for Solana (producer_solana.py)

```
import time
import json
import requests
from datetime import datetime
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

URL =
"https://api.coingecko.com/api/v3/simple/price?ids=solana&vs_currencies=usd&include_24hr_vol=true&include
_24hr_change=true"
TOPIC = "crypto-solana"

print("--- Starting Solana Kafka Producer ---")

while True:
    try:
        res = requests.get(URL, timeout=10)
        if res.status_code == 200:
            data = res.json().get("solana", {})
            if "usd" in data:
                payload = {
                    "crypto_id": "solana",
                    "symbol": "SOL",
                    "price_usd": float(data["usd"]),
                    "volume_24h": float(data.get("usd_24h_vol", 0.0)),
                    "change_24h": round(float(data.get("usd_24h_change", 0.0)), 2),
                    "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
            }
            producer.send(TOPIC, payload)
            print(f"[SOL Sent] {payload}")
        else:
            print(f"[SOL Warning] Missing payload data: {res.text}")
    elif res.status_code == 429:
        print("[SOL Rate Limit] CoinGecko rate limit reached. Waiting...")
    else:
        print(f"[SOL Error] HTTP {res.status_code}")
except Exception as e:
    print(f"[SOL Connection Error] {e}")

time.sleep(10)
```

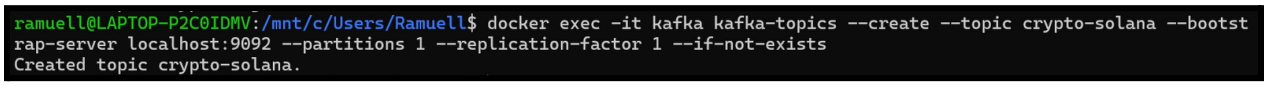


Figure 2. Solana Kafka Topic Creation

The terminal output confirms the successful creation of the crypto-solana Kafka topic. Like the Dogecoin topic, it uses one partition and a replication factor of one.

3. Stream Processing using Spark

Stream processing is implemented using PySpark in a micro-batch pattern. The application uses a Kafka consumer to poll the crypto-dogecoin and crypto-solana topics with a five-second timeout. Each polled batch is converted into a PySpark DataFrame using a predefined six-field schema, displayed in the console log, and inserted into Apache Cassandra using prepared CQL statements. Each batch runs as Spark jobs that can be monitored in the Spark Web UI (Apache Software Foundation, n.d.-c). The subtitle shown on the web interface (Figure 8) describes the pipeline as a whole; the Spark component itself processes Kafka messages in micro-batches, as described above.

Listing 3. Spark Micro-Batch Processing Script (spark_streaming.py)

```
import json
import time
from datetime import datetime
from kafka import KafkaConsumer
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
from cassandra.cluster import Cluster

print("Initializing Spark & Cassandra...")
cluster = Cluster(['127.0.0.1'], port=9042)
session = cluster.connect('crypto_stream')

prepared_stmt = session.prepare("""
    INSERT INTO market_data (crypto_id, symbol, price_usd, volume_24h, change_24h, timestamp)
    VALUES (?, ?, ?, ?, ?, ?)
""")

spark = SparkSession.builder \
    .appName("CryptoStreamingPipeline") \
    .master("local[*]") \
    .config("spark.ui.port", "4040") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

schema = StructType([
    StructField("crypto_id", StringType(), True),
    StructField("symbol", StringType(), True),
    StructField("price_usd", DoubleType(), True),
    StructField("volume_24h", DoubleType(), True),
    StructField("change_24h", DoubleType(), True),
    StructField("timestamp", StringType(), True)
])

consumer = KafkaConsumer(
    'crypto-dogecoin', 'crypto-solana',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='latest',
    enable_auto_commit=True,
    group_id='spark-cassandra-pipeline',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

print("Starting Spark Micro-Batch Processing Loop... (Press Ctrl+C to stop)")
print("Spark Web UI available at: http://localhost:4040")
batch_id = 0

try:
    while True:
        raw_msgs = consumer.poll(timeout_ms=5000)
        records = []
        for tp, msgs in raw_msgs.items():
```

```
            for msg in msgs:
                records.append(msg.value)

        if records:
            # Create Spark DataFrame from micro-batch
            df = spark.createDataFrame(records, schema=schema)
            count = df.count()

            print(f"\n==========================================")
            print(f"--- Processing Spark Micro-Batch {batch_id} ({count} rows) ---")
            df.show(truncate=False)

            # Insert parsed batch into Cassandra
            for row in df.collect():
                # Parse timestamp string to datetime object for Cassandra
                try:
                    ts_val = datetime.strptime(row['timestamp'], "%Y-%m-%d %H:%M:%S")
                except Exception:
                    ts_val = datetime.now()

                session.execute(prepared_stmt, (
                    row['crypto_id'],
                    row['symbol'],
                    float(row['price_usd']),
                    float(row['volume_24h']),
                    float(row['change_24h']),
                    ts_val
                ))

            print(f"[Spark Batch {batch_id}] Successfully persisted to Cassandra table -> market_data")
            print(f"==========================================\n")
            batch_id += 1
        else:
            time.sleep(1)

except KeyboardInterrupt:
    print("\nStopping streaming pipeline...")
finally:
    consumer.close()
    spark.stop()
    cluster.shutdown()
```

4. Data Storage using Cassandra

Apache Cassandra is used as the NoSQL storage layer for storing streaming cryptocurrency market data. According to the Apache Software Foundation (n.d.-a), keyspaces define data replication settings, while tables organize stored data through their schemas and primary keys.

```
ramuell@LAPTOP-P2C0IDMV:~/crypto_pipeline$ docker exec -it cassandra cqlsh -e "CREATE KEYSPACE IF NOT EXISTS crypto_stre
am WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};"
ramuell@LAPTOP-P2C0IDMV:~/crypto_pipeline$ docker exec -it cassandra cqlsh -e "CREATE TABLE IF NOT EXISTS crypto_stream.
market_data (crypto_id text, symbol text, price_usd double, volume_24h double, change_24h double, timestamp timestamp, P
RIMARY KEY ((crypto_id), timestamp)) WITH CLUSTERING ORDER BY (timestamp DESC);"
```

Figure 3. Cassandra Keyspace and Table Creation CQL Script

Figure 3 presents the CQL commands used to create the crypto_stream keyspace with SimpleStrategy and a replication factor of 1, and the market_data table inside it. The table defines six columns matching the extracted fields (crypto_id, symbol, price_usd, volume_24h, change_24h, and timestamp), with crypto_id as the partition key and timestamp as the clustering column in descending order.

```
CREATE TABLE crypto_stream.market_data (
    crypto_id text,
    timestamp text,
    change_24h double,
    price_usd double,
    symbol text,
    volume_24h double,
    PRIMARY KEY (crypto_id, timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC)
    AND additional_write_policy = '99p'
    AND allow_auto_snapshot = true
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32',
 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
    AND incremental_backups = true
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
```

Figure 4. Cassandra market_data Table Schema

Figure 4 defines the market_data table within the crypto_stream keyspace. The table uses crypto_id as the partition key and timestamp as the clustering column in descending order, allowing records to be organized by cryptocurrency and recent timestamps. This schema supports queries that retrieve the latest streaming market metrics for each cryptocurrency (Apache Software Foundation, n.d.-a).

Listing 4. Cassandra Table Definition CQL Script

```
CREATE KEYSPACE IF NOT EXISTS crypto_stream WITH replication = {'class': 'SimpleStrategy',
'replication_factor': 1};
CREATE TABLE IF NOT EXISTS crypto_stream.market_data (
    crypto_id text,
    symbol text,
    price_usd double,
    volume_24h double,
    change_24h double,
    timestamp timestamp,
    PRIMARY KEY ((crypto_id), timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

5. Web Application using Flask

A web user interface built with the Flask framework provides near real-time visualization of stored cryptocurrency metrics. The backend server script app.py connects to the Cassandra cluster, queries the 25 most recent records for Dogecoin and Solana from the market_data table, and renders them in a formatted table layout defined in index.html (Pallets, n.d.). The resulting page is shown in Figures 8 and 9 and displays the stored records as of each page load.

Listing 5. Flask Web Application Script (app.py)

```
from flask import Flask, render_template
from cassandra.cluster import Cluster

app = Flask(__name__)

cluster = Cluster(['127.0.0.1'], port=9042)
session = cluster.connect('crypto_stream')

@app.route('/')
def index():
    # we fetch dogecoin data :)
    doge_rows = session.execute(
        "SELECT crypto_id, symbol, price_usd, volume_24h, change_24h, timestamp "
        "FROM market_data WHERE crypto_id = 'dogecoin' "
        "ORDER BY timestamp DESC LIMIT 25"
    )

    # we fetch solana data :)
    sol_rows = session.execute(
        "SELECT crypto_id, symbol, price_usd, volume_24h, change_24h, timestamp "
        "FROM market_data WHERE crypto_id = 'solana' "
```

```
        "ORDER BY timestamp DESC LIMIT 25"
    )

    return render_template('index.html', doge=list(doge_rows), sol=list(sol_rows))

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)
```

The HTML template index.html provides the visual presentation layer, structuring streaming market updates into clean table elements for user monitoring.

6. Main Workflow and Screenshots

6.1 Producers

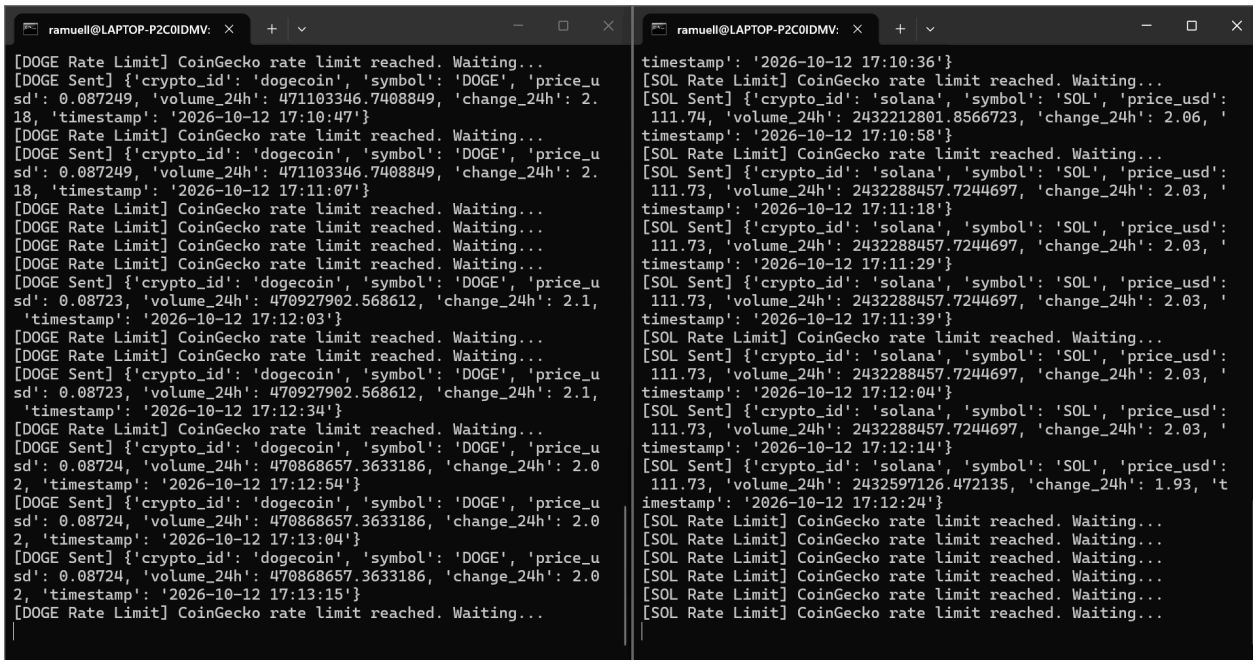


Figure 5. Dogecoin and Solana Kafka Producers Terminal Execution

As demonstrated in Figure 5, both producer processes operate concurrently in isolated terminal windows. The Dogecoin producer publishes messages to crypto-dogecoin (left), while the Solana producer publishes messages to crypto-solana (right), maintaining strict segregation of streaming event topics. The terminal output also shows “CoinGecko rate limit reached.

Waiting...” messages between successful sends. This occurs when the CoinGecko API temporarily rejects a request because too many were made, and the producers handle it by logging the event and continuing the loop instead of crashing. As a result, successful messages arrive at irregular intervals (for example, the Dogecoin timestamps of 17:12:03, 17:12:34, and 17:12:54), while the stream continues without interruption.

6.2 Consumers

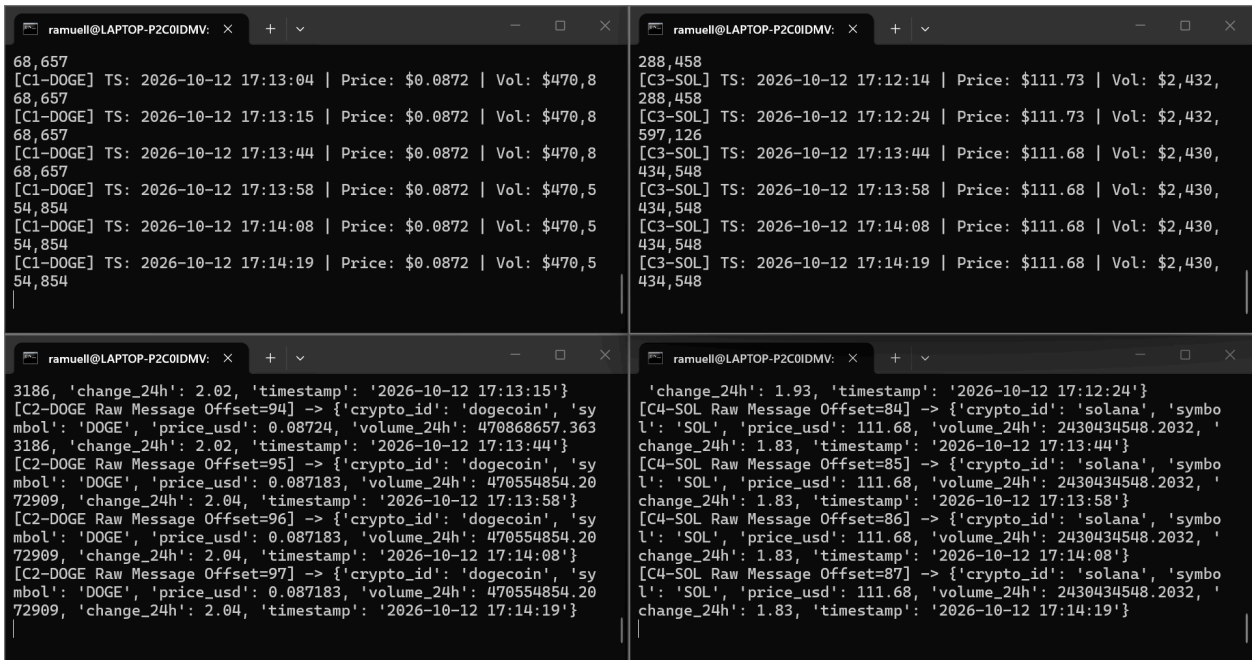


Figure 6. Dogecoin and Solana Kafka Consumers Terminal Execution

Figure 6 displays four Kafka consumers running in separate terminals, two per topic. Consumers 1 (C1-DOGE) and 3 (C3-SOL) each read their topic independently and print a formatted line per message showing the timestamp, price, and 24-hour volume. Consumers 2 (C2-DOGE) and 4 (C4-SOL) read the same topics and print each raw JSON message together with its partition offset. Together, they confirm that both topics are receiving the producers’ messages and that each stream stays separate, with Dogecoin messages appearing only in the Dogecoin consumers and Solana messages only in the Solana consumers.

6.3 Spark Application

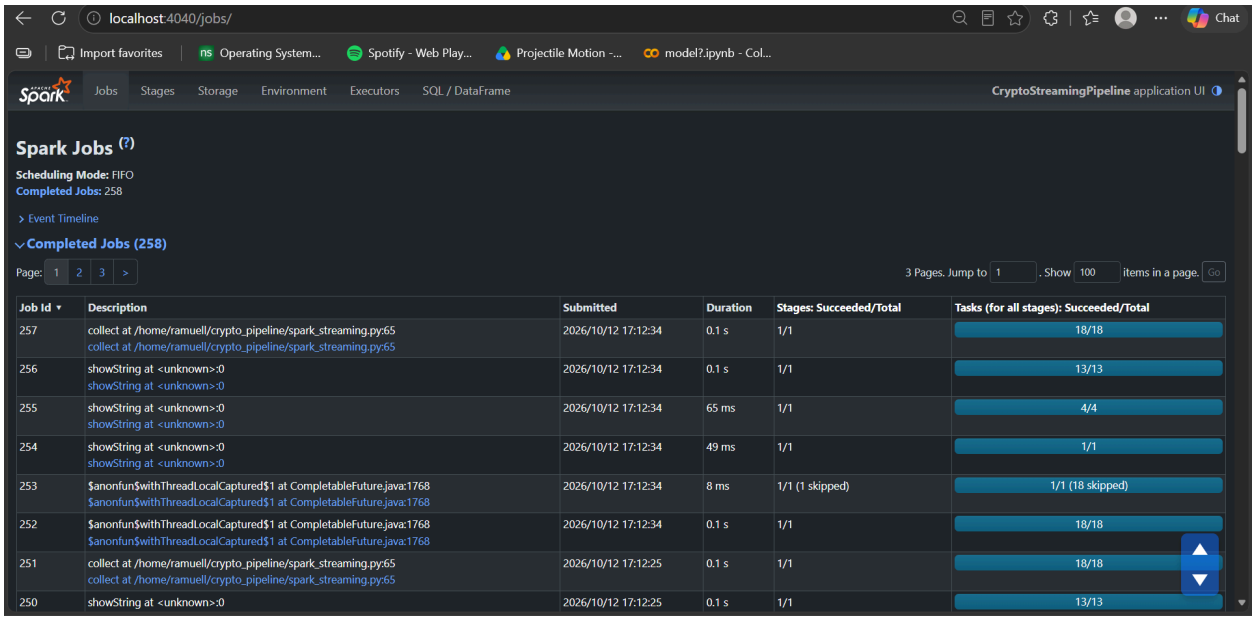


Figure 7. Apache Spark Web UI Completed Jobs

Figure 7 shows the Apache Spark Web UI (localhost:4040) for the CryptoStreamingPipeline application. The Completed Jobs table lists 258 successful jobs, mainly the showString and collect actions generated by the df.show() and df.collect() calls in spark_streaming.py. New jobs appear in groups at the same timestamps (for example, 17:12:34), with one group created for each micro-batch of Kafka messages processed. This confirms that Spark is actively processing the incoming data.

6.4 Flask Website

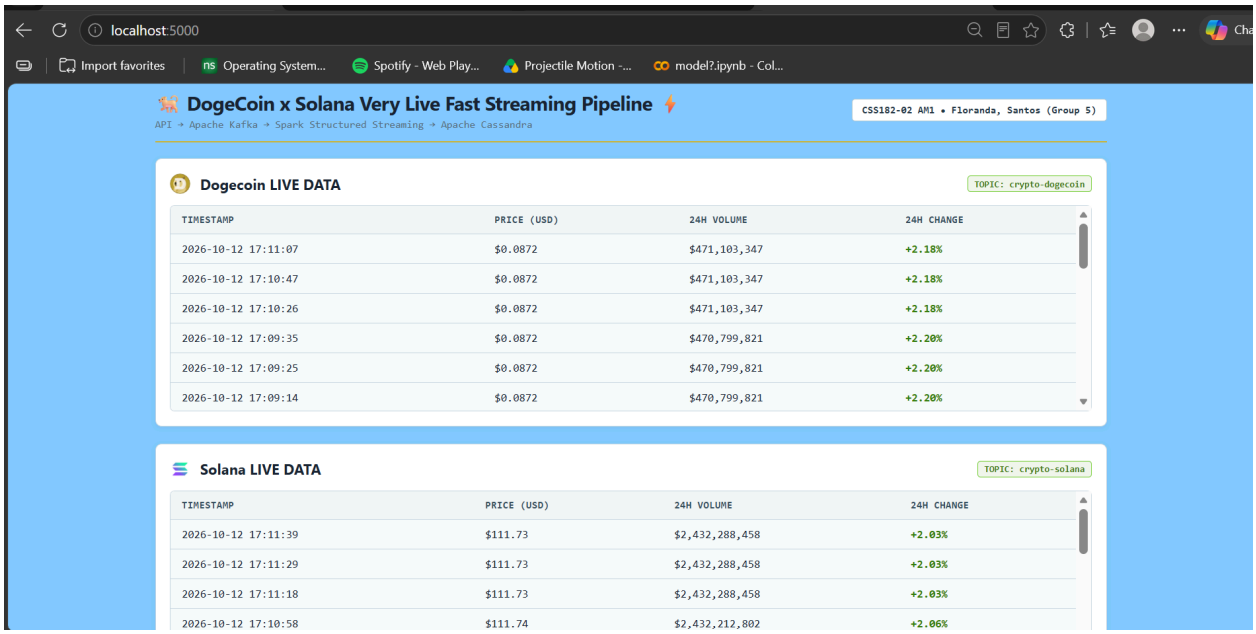


Figure 8. Flask Web Application Interface (Top View)

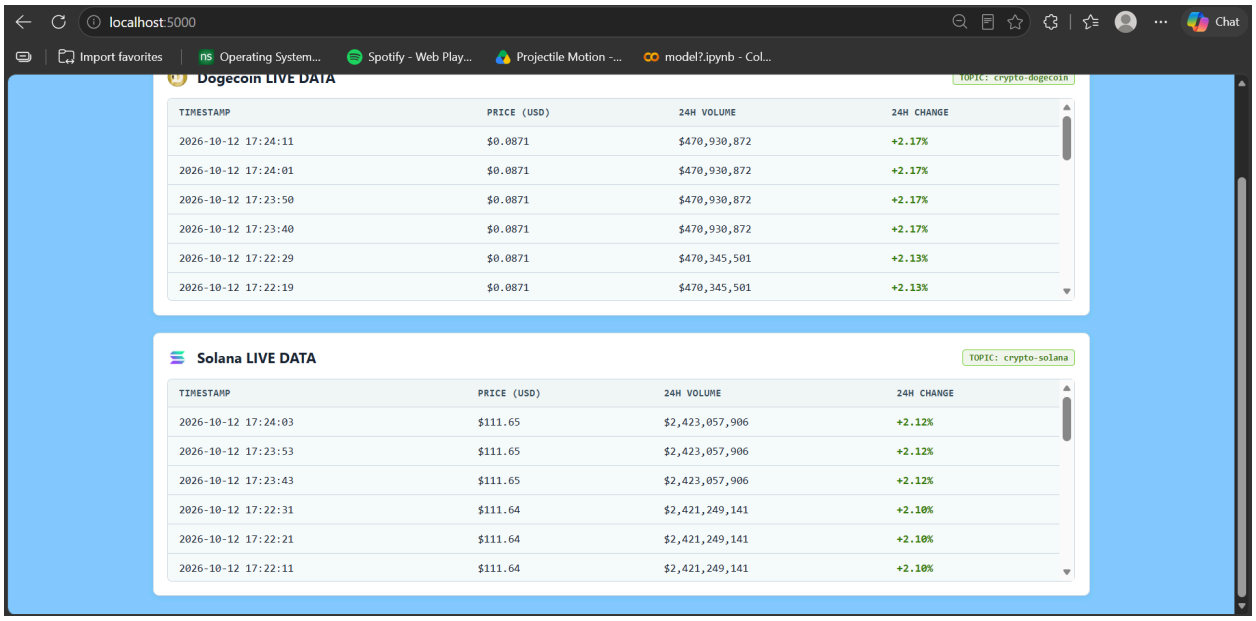


Figure 9. Flask Web Application Interface (Bottom View)

Figures 8 and 9 show the Flask web application (localhost:5000), which reads the stored records from the Cassandra market_data table and displays them in two scrollable tables, one for

Dogecoin and one for Solana, each labeled with its Kafka topic. Each table lists the timestamp, price, 24-hour volume, and 24-hour change, with the newest record first. Figure 9 was captured later than Figure 8 and shows newer timestamps (17:22 to 17:24, compared with 17:09 to 17:11 in Figure 8), showing that the page displays new records as they are stored in Cassandra.

7. Video Demonstration Link

https://youtu.be/IPxghgMYc6k?si=prcq4b-eCAaFFGU9

8. Student Learnings and Contributions

8.1 Floranda, Ramuell Franz C.

Project Contributions: Built and ran the Kafka producers, Spark processing script, Cassandra setup and Flask application; led the video demonstration and monitored the database during the demo.

Learnings: I find it very cool that technology nowadays can do much more grand stuff than just using it for documents and performing basic calculations. By taking the Data Science Course, I have learned how to process real-time data using Apache Kafka, Spark, and Cassandra. To process real-time data from other sources, we can use their public API to place the extracted data in our database, analyze the extracted data, and use them for our own web application while integrating Flask. This process is way more advanced than simply creating a document file or process basic calculations. By completing this project, I gained more advanced techniques on how to process data and create a simple system while using the extracted data.

8.2 Santos, Lebbeus Ryan V.

Project Contributions: Prepared the project documentation, including workflow explanations, code listings, figure captions and references; proofread the document and verified citations and links.

Learnings: In Data Science 2, I learned that data science involves more than just programming in Python and analyzing datasets. Compared to Introduction to Data Science and Data Science 1, which focused on the fundamentals, this specialization introduced me to real-time data processing and how different technologies work together, such as Kafka, Spark, Cassandra, and Flask. I gained a better understanding of how data can be collected from live sources, processed continuously, stored in a database, and displayed through a web application. This experience helped me appreciate the importance of data pipelines and how the concepts I learned in my previous courses can be applied to a more complete and practical system.

9. References

Apache Software Foundation. (n.d.-a). *Apache Cassandra documentation*. https://cassandra.apache.org/doc/latest/cassandra/cql/

Apache Software Foundation. (n.d.-b). *Apache Kafka documentation*. https://kafka.apache.org/documentation/

Apache Software Foundation. (n.d.-c). *Monitoring and instrumentation*. https://spark.apache.org/docs/latest/monitoring.html

CoinGecko. (n.d.). *CoinGecko API documentation*. https://www.coingecko.com/en/api/documentation

Pallets. (n.d.). *Flask documentation*. https://flask.palletsprojects.com/

10. Appendices

Appendix A. HTML User Interface Template (index.html)

https://drive.google.com/file/d/1BTx8ePrFyl_9pPYSeeaaJ3cUc7IFPQxo/view?usp=sharing

Appendix B. Full Project Repository and Scripts

https://drive.google.com/drive/folders/1kk790a6Dc2ATWxANehv3Bmbw8rQkSGIT?usp=drive_link

Appendix C. YouTube Demonstration

https://youtu.be/IPxghgMYc6k?si=prcq4b-eCAaFFGU9